

SQL – DQL

(Data Query Language)

DQL, or Data Query Language, is the part of SQL used to retrieve data from a database.

The main DQL operations are:

1. SELECT—Specifies which columns or expressions to retrieve.

FROM—Specifies the table or tables to retrieve data from.

2. DISTINCT—Removes duplicate rows from the result.

3. WHERE, LOGICAL OPERATORS, BETWEEN—Filters rows based on a condition.

4. IN, LIKE and Wildcards—Checks whether a value matches any value in a list or subquery.

5. Working With NULL

6. ORDER BY(ASC, DESC)—Sorts the query results.

7. LIMIT—Restricts the number of rows returned in MySQL.

8. Aggregate Functions -

9. GROUP BY—Groups rows that have the same values.

10. HAVING—Filters groups created by GROUP BY

10. JOIN (INNER JOIN)—Combines related rows from two or more tables.

LEFT JOIN—Returns all rows from the left table and matching rows from the right table.

RIGHT JOIN—Returns all rows from the right table and matching rows from the left table.

Joining Multiple Tables —

AS —Gives a temporary alias to a column or table.

LIKE — Searches for a pattern in text.

IS NULL — Checks whether a value is NULL.

IS NOT NULL — Checks whether a value is not NULL.

WITH — Defines a Common Table Expression (CTE) for use in a query.

1. SELECT and FROM

Note: Use 'SQL DQL Scratchpad.sql' to create the database, tables, and insert values.

To retrieve all columns, use the asterisk *:

```
SELECT *  
FROM Student;
```

Instead of displaying every column in a table, we can specify only the columns we want.

Example:

```
SELECT ID, Name, Major  
FROM Student;
```

This retrieves only the StudentID, Name, and Major columns.

2. DISTINCT

The DISTINCT keyword removes duplicate rows from the query result.

Example:

```
SELECT Major  
FROM Student;
```

This might produce:

Computer Science
Biology
Business
Computer Science
Mathematics
Business
Computer Science
Biology
Mathematics
Computer Science
Business
Biology

Using DISTINCT:

```
SELECT DISTINCT Major  
FROM Student;
```

would produce:

Computer Science
Biology
Business
Mathematics

3. WHERE

The WHERE clause is used to select only rows that meet a particular condition.

Example:

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science';
```

Only students whose major is Computer Science will be displayed.
Common comparison operators include:

Another Example:

```
SELECT StudentID  
FROM Enrollment  
WHERE Grade <= 'B';
```

Operator	Meaning
=	Equal to
<>	Not equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical Operators

Multiple conditions can be combined using logical operators.

The most common are:

1. AND
2. OR
3. NOT

AND

Both conditions must be true.

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
AND GPA >= 3.0;
```

OR

At least one condition must be true.

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
OR Major = 'Biology';
```

NOT

NOT reverses a condition.

```
SELECT *  
FROM Student  
WHERE NOT Major = 'Computer Science';
```

Parentheses can be used when combining several conditions.

```
SELECT *  
FROM Student  
WHERE GPA >= 3.0  
AND (Major = 'Computer Science'  
    OR Major = 'Biology');
```

BETWEEN

BETWEEN checks whether a value falls within a range (inclusive).

Example:

```
SELECT *  
FROM Student  
WHERE GPA BETWEEN 3.0 AND 4.0;
```

This includes both 3.0 and 4.0.

The equivalent condition would be:

```
SELECT *  
FROM Student  
WHERE GPA >= 3.0  
AND GPA <= 4.0;
```

4. IN

The IN operator checks whether a value matches one of several possible values.

Instead of writing:

```
SELECT *  
FROM Student  
WHERE Major = 'Computer Science'  
OR Major = 'Biology'  
OR Major = 'Business';
```

we can write:

```
SELECT *  
FROM Student  
WHERE Major IN ('Computer Science',  
                'Biology',  
                'Business');
```

IN is often easier to read when several possible values are being checked.

5. LIKE and Wildcards

The LIKE operator is used for pattern matching.
Two commonly used wildcard characters are:

Wildcard	Meaning
%	Zero or more characters
_	Exactly one character

Example:

```
SELECT *  
FROM Student  
WHERE Name LIKE 'A%';
```

This retrieves students whose names begin with A.

Examples:

```
SELECT *  
FROM Student  
WHERE Name LIKE '%son';
```

Retrieves names ending in son.

```
SELECT *  
FROM Student  
WHERE Name LIKE '%le%';
```

Retrieves names containing le.

```
SELECT *  
FROM Student  
WHERE Name LIKE 'J_n%';
```

Retrieves names that starts with 'J' and the third letter is an 'n' .
Examples might include:
'Jun' and 'Jan'

Working with NULL

NULL means that a value is missing, unknown, or not available.

To check for **NULL**, use:
IS NULL

Example:

```
SELECT *  
FROM Student  
WHERE Major IS NULL;
```

To find values that are not NULL:

```
SELECT *  
FROM Student  
WHERE Major IS NOT NULL;
```

Do not write:

WHERE Major = NULL;

This will not work as expected because **NULL** represents an unknown value.

5. ORDER BY

The ORDER BY clause sorts the rows returned by a query.

Example:

```
SELECT *  
FROM Student  
ORDER BY Name;
```

By default, the order is ascending.

You may explicitly write:

```
ORDER BY Name ASC;
```

For descending order:

```
ORDER BY Name DESC;
```

You can also sort using multiple columns.

```
SELECT *  
FROM Student  
ORDER BY Major, Name;
```

The rows will first be sorted by Major. Students within the same major will then be sorted by Name.

6. LIMIT

In MySQL, LIMIT restricts the number of rows returned.

Example:

```
SELECT *  
FROM Student  
LIMIT 5;
```

Only the first five rows will be returned.

It is often combined with ORDER BY.

```
SELECT *  
FROM Student  
ORDER BY GPA DESC  
LIMIT 5;
```

This displays the five students with the highest GPAs.

7. Aggregate Functions

Aggregate functions perform calculations using multiple rows and return a summarized result.

Common aggregate functions include:

COUNT()

Counts rows.

```
SELECT COUNT(*)  
FROM Student;
```

SUM()

Adds numeric values.

```
SELECT SUM(Credits)  
FROM Course;
```

AVG()

Calculates an average.

```
SELECT AVG(GPA)  
FROM Student;
```

MIN()

Finds the smallest value.

```
SELECT MIN(GPA)  
FROM Student;
```

MAX()

Finds the largest value.

```
SELECT MAX(GPA)  
FROM Student;
```

To find the name of the student with the highest GPA:

```
SELECT Name  
FROM Student  
WHERE GPA = (  
    SELECT MAX(GPA)  
    FROM Student);
```

8. GROUP BY

GROUP BY creates groups of rows that share the same value. Aggregate functions can then be applied to each group.

Example:

```
SELECT Major,  
       COUNT(*) AS NumberOfStudents  
FROM Student  
GROUP BY Major;
```

The result might look like:

Computer Science	15
Biology	10
Business	12

Instead of counting all students together, SQL counts the students within each major.

Another example:

```
SELECT Major,  
       ROUND(AVG(GPA), 2) AS 'Average GPA'  
FROM Student  
GROUP BY Major;
```

9. HAVING

HAVING filters grouped results. It is commonly used together with GROUP BY.

Example:

```
SELECT Major,  
       COUNT(*) AS NumberOfStudents  
FROM Student  
GROUP BY Major  
HAVING COUNT(*) >= 3;
```

Only majors with at least 3 students will be displayed.

A useful distinction is:

WHERE → filters rows before grouping

HAVING → filters groups after GROUP BY

Example:

```
SELECT Major, AVG(GPA)  
FROM Student  
WHERE GPA IS NOT NULL  
GROUP BY Major  
HAVING AVG(GPA) >= 3.0;
```

10. JOIN

A JOIN combines rows from two or more related tables.

Suppose we have:

Student

StudentID

Name

Enrollment

StudentID

CourseID

We can combine the tables using:

```
SELECT Student.ID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
JOIN Enrollment  
  ON Student.ID = Enrollment.StudentID;
```

The ON clause describes how rows in the two tables are related.

JOIN without another keyword normally means INNER JOIN.

Therefore:

JOIN

and

INNER JOIN

mean the same thing.

An inner join returns rows only when a matching row exists in both tables.

LEFT JOIN

A LEFT JOIN returns:
all rows from the left table matching rows from the right table. If there is no matching row, the columns from the right table contain NULL.

Example:

```
SELECT Student.ID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
LEFT JOIN Enrollment  
ON Student.ID = Enrollment.StudentID;
```

All students will appear, even students who are not enrolled in a course. The table written before LEFT JOIN is the **left table**.

In this example:

Student
is the left table.

For example: Show all courses that have no students enrolled.

```
SELECT Course.CourseID,  
       Course.Name  
FROM Course  
LEFT JOIN Enrollment  
ON Course.CourseID = Enrollment.CourseID  
WHERE Enrollment.StudentID IS NULL;
```

RIGHT JOIN

A RIGHT JOIN does the opposite of a LEFT JOIN.

It returns:

all rows from the right table

matching rows from the left table

Example:

```
SELECT Student.StudentID,  
       Student.Name,  
       Enrollment.CourseID  
FROM Student  
RIGHT JOIN Enrollment  
ON Student.StudentID = Enrollment.StudentID;
```

The table written after RIGHT JOIN is the right table.

In many situations, a RIGHT JOIN can be rewritten as a LEFT JOIN simply by reversing the table order. Because of this, many developers primarily use LEFT JOIN.

Joining More Than Two Tables

Queries frequently combine several tables.

For example:

Student

Enrollment

Course

A student enrolls in a course through the Enrollment table. The query may be:

```
SELECT Student.Name AS StudentName,  
       Course.Name AS CourseName  
FROM Student  
JOIN Enrollment  
  ON Student.ID = Enrollment.StudentID  
JOIN Course  
  ON Enrollment.CourseID = Course.ID;
```

This query follows the relationships:

Student

|

Enrollment

|

Course

Subqueries

A **subquery** is a query contained inside another query.

For example:

```
SELECT ID, Name  
FROM Student  
WHERE ID IN (  
    SELECT StudentID  
    FROM Enrollment  
);
```

The inner query:

```
SELECT StudentID  
FROM Enrollment
```

runs as part of the larger query.

The outer query then uses its result.

The query containing the subquery is sometimes called the **outer query** or **main query**.

Common Table Expressions (CTE)

A **Common Table Expression**, or **CTE**, creates a temporary named query result.

A CTE begins with the keyword: WITH

Example:

```
WITH CourseCounts AS (  
  SELECT FacultyID,  
    COUNT(*) AS NumCourses  
  FROM Course  
  GROUP BY FacultyID  
)  
SELECT *  
FROM CourseCounts;
```

CourseCounts exists only while the query is being executed.

Without CTE, the SQL on the left will be written as:

```
SELECT FacultyID,  
  COUNT(*) AS NumCourses  
FROM Course  
GROUP BY FacultyID;
```